

PRACTICAL : WEEK 7

Task - 1:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main()
{
    int stack_var = 30;
    int *heap_var = (int *)malloc(sizeof(int));

    *heap_var = 40;

    printf("Memory Addresses:\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address    : %p\n", (void *)heap_var);
    printf("Stack address   : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano memory.c
Radhakrishna@04:~# gcc memory.c -o memory
Radhakrishna@04:~# ./memory
Memory Addresses:
Code address   : 0x5eb3901a61c9
Global address : 0x5eb3901a9010
Static address : 0x5eb3901a9014
Heap address   : 0x5eb3967132a0
Stack address  : 0x7ffe0h8bbdcc
Radhakrishna@04:~#
```

Task - 2:

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int global_var = 100;
static int static_var = 200;

int main()
{
    int stack_var = 300;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 400;

    printf("Process is running...\n");
    printf("PID: %d\n", getpid());

    printf("Global variable address: %p\n", (void *)&global_var);
    printf("Static variable address: %p\n", (void *)&static_var);
    printf("Heap variable address: %p\n", (void *)heap_var);
    printf("Stack variable address: %p\n", (void *)&stack_var);

    printf("\nProcess will remain active for 60 seconds.\n");

    sleep(60);

    free(heap_var);

    return 0;
}
```

OUTPUT:

```
Radhakrishna@04:~# nano memorymap.c
Radhakrishna@04:~# gcc memorymap.c -o memorymap
Radhakrishna@04:~# ./memorymap
Process is running...
PID: 458
Global variable address: 0x55a608c6c010
Static variable address: 0x55a608c6c014
Heap variable address: 0x55a62c8532a0
Stack variable address: 0x7fffeeefd29ac

Process will remain active for 60 seconds.
Radhakrishna@04:~# gcc memory.c -o memory
Radhakrishna@04:~# gcc memory.c -o memorymap
Radhakrishna@04:~# ./memorymap
Memory Addresses:
Code address : 0x63faf89ab1c9
Global address : 0x63faf89ae010
Static address : 0x63faf89ae014
Heap address : 0x63fb063a82a0
Stack address : 0x7fffc86c2a5c
Radhakrishna@04:~#|
```